Московский государственный университет имени М.В. Ломоносова Факультет вычислительной математики и кибернетики Кафедра автоматизации систем вычислительных комплексов

Ефанов Михаил Михайлович

Алгоритм управления перегрузкой на основе пропускной способности и задержки в условиях применения пакетной балансировки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Научный руководитель:
д.ф.-м.н., профессор, чл.-корр. РАН Смелянский Руслан Леонидович

Научный консультант:
к.ф.-м.н.,ассистент Степанов Евгений Павлович

Москва, 2026

Аннотация

Работа посвящена разработке алгоритма управления перегрузкой на основе пропускной способности и задержки (BBR), адаптированного для работы в условиях пакетной балансировки. Пакетная балансировка значительно повышает производительность и надежность сетевых сервисов за счет горизонтального масштабирования, однако классические алгоритмы управления перегрузкой не адаптированы для совместной работы с ней.

Основные проблемы, возникающие при использовании пакетной балансировки, включают переупорядочивание пакетов и перегрузку каналов, которая приводит к ложным сигналам перегрузки и снижению скорости передачи данных. В работе проведен обзор существующих решений, направленных на повышение устойчивости алгоритмов управления перегрузкой к переупорядочиванию пакетов и перегрузке каналов: TCP-NCR, Lazy TCP (LTCP) и MSwift.

На основе сравнительного анализа для модификации алгоритма BBR выбран подход, используемый в MSwift, который обеспечивает устойчивость как к переупорядочиванию пакетов, так и к перегрузке каналов. Разработан стенд для моделирования пакетной балансировки в системе ns3, позволяющий исследовать поведение алгоритмов в условиях асимметрии каналов.

Оглавление

Введение 3
1. Постановка задачи 5
2. Обзор существующих решений 5
   2.1. Основные проблемы при пакетной балансировке 5
      2.1.1. Переупорядочивание пакетов 5
      2.1.2. Деградация отдельных путей 6
   2.2. BBR как базовый алгоритм управления перегрузкой 6
   2.3. TCP-NCR: динамический порог реакции на duplicate ACK 6
   2.4. Lazy TCP (LTCP): “ленивый триггер” и “упрямое окно” 7
   2.5. MSwift: устойчивая к пакетной балансировке агрегация сигналов (медиана) 8
   2.6. Выводы по обзору и выбор направления для модификации BBR 9
Литература 10

Введение

Современные компьютерные сети характеризуются высокой динамикой нагрузки, неоднородностью каналов и всё более широким применением многопутевой передачи. На практике трафик одного транспортного соединения может проходить по нескольким параллельным маршрутам — как из-за особенностей маршрутизации и инженерии трафика в сети, так и из-за применения механизмов балансировки нагрузки. При этом балансировка может выполняться не только по потокам, но и на уровне отдельных пакетов (per-packet load balancing, packet spraying), когда последовательные пакеты одного и того же соединения целенаправленно распределяются по разным путям. Такая стратегия потенциально повышает использование доступной пропускной способности совокупности путей и улучшает устойчивость передачи к локальным перегрузкам и отказам, но одновременно меняет свойства наблюдаемых транспортом сигналов состояния сети [1].

Классические алгоритмы управления перегрузкой TCP исторически опираются на интерпретацию потери пакета (loss-based) как основного признака перегрузки. Эта эвристика была разумной в условиях ограниченных буферов и относительно простых путей, однако в современных сетях приводит к систематическим проблемам. При больших буферах на узких местах loss-based управление перегрузкой склонно формировать устойчивые очереди и увеличивать задержки, а при потерях, не связанных с перегрузкой, — неоправданно снижать скорость, недоиспользуя доступную ёмкость канала. На этом фоне получили развитие подходы, основанные не на потерях, а на моделировании физических ограничений пути: пропускной способности узкого места (BtlBw) и задержки распространения (RTprop). Алгоритм BBR является представителем этого направления: он оценивает BtlBw и RTprop, стремится поддерживать объём данных «в полёте» (отправленные пакеты на, которые еще не получен ACK) около BDP = BtlBw × RTprop, а темп передачи задаёт через задержку между отправкой следующего пакета (pacing). Такой подход позволяет одновременно добиваться высокой полезной пропускной способности и удерживать задержки на низком уровне, что подтверждается опытом крупномасштабного применения в реальных сервисах и на разнообразных типах сетей, включая мобильные [2].

Однако применение многопутевой передачи и особенно пакетной балансировки усложняет задачу управления перегрузкой. Транспортный протокол перестаёт наблюдать «один путь» с единственными характеристиками задержки и пропускной способности; вместо этого он получает поток обратной связи, являющийся смесью нескольких путей, которые могут различаться по задержкам, текущим очередям и доступной скорости. Даже если изначально пути близки по параметрам, в реальной сети неизбежны временные асимметрии: локальные перегрузки, изменения маршрута, сбои и деградации отдельных каналов, приводящие к увеличению

задержки или потере части пакетов на одном из путей. В таких условиях пакеты могут приходить не по порядку, что порождает дубликатные ACK и другие аномальные наблюдения, которые традиционный TCP склонен трактовать как перегрузку. Это приводит к неоправданному снижению скорости передачи и деградации производительности — иногда до состояния, которое можно описать как коллапс пропускной способности при наличии даже небольшой доли перегруженных путей из общего числа путей, между которыми реализована балансировка пакетов [3].

Проблема устойчивости TCP к переупорядочению пакетов рассмотрена в [4]: в этой работе введено понятие «устойчивости к неперегрузочным событиям» (Non-Congestion Robustness) и предложен подход, в котором порог срабатывания быстрой ретрансляции по duplicate ACK не фиксирован, а динамически увеличивается в зависимости от состояния соединения. Это позволяет лучше отличать истинную потерю от переупорядочения и уменьшает число избыточных повторных передач, хотя вводит компромисс в виде замедления реакции на реальную перегрузку. В работе [5] предложены более специализированные механизмы: Lazy TCP откладывает реакцию на dupACK («lazy trigger») и по-разному реагирует на единичные и последовательные потери, стремясь не снижать скорость из-за событий, которые с высокой вероятностью не являются перегрузкой.

Однако устойчивости только к переупорядочиванию пакетов часто недостаточно. Современные алгоритмы управления перегрузкой используют разные сигналы, например задержку (delay-based). В условиях пакетной балансировки эти сигналы начинают поступать как смесь измерений по разным путям и могут содержать выбросы из-за перегруженного или более длинного пути. Показано, что для широкого класса алгоритмов, использующих AIMD-подобные принципы, при наличии перегруженного пути с вероятностью попадания на него q пропускная способность может ухудшаться по закону порядка $\Theta(1/\sqrt{q})$; причём это проявляется для различных CCA и сохраняется даже у вариантов, устойчивых к переупорядочению [3]. В качестве более общего принципа предложено применять устойчивую к подобным выбросам агрегацию многопутевых сигналов, например использовать медиану по окну недавних измерений вместо реакции на «последнее» наблюдение [3].

Таким образом, актуальность темы данной дипломной работы определяется следующими обстоятельствами:

- распространённостью и практической значимостью алгоритмов управления перегрузкой, ориентированных на пропускную способность и задержку, таких как BBR, позволяющих приблизиться к высокой утилизации канала при малых очередях [2];

- растущей ролью пакетной балансировки и других форм многопутевой доставки, которые могут повышать использование доступной суммарной ёмкости, но провоцируют переупорядочение и делают транспортные измерения смешанными по нескольким путям [1];
- наличием фундаментальных эффектов деградации производительности CCA в многопутевом per-packet сценарии и необходимостью более робастной обработки сигналов перегрузки, чем простая защита от dupACK [3];
- существованием частичных решений (динамический порог dupACK, «ленивые» реакции), которые полезны, но не закрывают полностью задачу устойчивого управления при многопутевой смешанной обратной связи [3–5]

1. Постановка задачи

Целью дипломной работы является повышение скорости транспортного соединения при сохранении низкой и предсказуемой задержки за счёт разработки модификации алгоритма BBR, адаптированной к работе в условиях пакетной балансировки.

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести обзор существующих алгоритмов управления перегрузкой, направленных на использование совместно с пакетной балансировкой и выявить причины уменьшения производительности CCA в условиях применения пакетной балансировки.
- Разработать модификации алгоритма BBR.
- Провести сравнительный анализа базового и модифицированного BBR.

2. Обзор существующих решений

2.1. Основные проблемы при пакетной балансировке

2.1.1. Переупорядочивание пакетов

При распылении пакетов по нескольким маршрутам пакеты одного соединения могут приходить к получателю не по порядку. Получатель в этом случае обычно отвечает **дубликатными ACK** (dupACK) — это ACK, который повторно подтверждает один и тот же номер байта, потому что следующий байт (или сегмент) ещё не получен. Отправитель может ошибочно принять серию dupACK за потерю и запустить механизмы восстановления, что снижает скорость передачи, хотя перегрузки могло не быть [1].

Будем считать транспортный агент **устойчивым к переупорядочиванию**, если переупорядочивание само по себе не приводит к уменьшению скорости передачи (то есть не запускает лишнее снижение скорости только из-за порядка прихода пакетов) [5].

2.1.2. Деградация отдельных путей

В per-packet сценарии достаточно, чтобы часть путей временно стала хуже (меньше пропускная способность, выше задержка или больше потерь), чтобы измерения на стороне транспорта начали содержать **выбросы** — редкие «плохие» измерения, нехарактерные для большинства путей. Если алгоритм управления перегрузкой реагирует на такие выбросы как на общую перегрузку всего соединения, он снижает скорость непропорционально сильно и недоиспользует суммарную емкость.

Для широкого класса AIMD-подобных алгоритмов показан эффект деградации при вероятности попадания пакета на перегруженный путь `q`: пропускная способность может ухудшаться порядка `Θ(1/√q)` [3]. Важно, что это относится не только к классическому TCP, но и к современным CCA, в том числе использующим задержку, и может сохраняться даже при добавлении устойчивости к переупорядочиванию [3].

Будем считать транспортный агент **устойчивым к деградации отдельных путей**, если наличие небольшой доли «плохих» путей не приводит к заметному уменьшению скорости на всём соединении (в идеале — скорость падает только настолько, насколько неизбежно из-за реально потерянных пакетов на плохих путях) [3].

2.2. BBR как базовый алгоритм управления перегрузкой

BBR относится к модельно-ориентированным CCA: вместо потерь как основного сигнала перегрузки он оценивает пропускную способность узкого места `BtlBw` и задержку распространения `RTprop`, после чего управляет:

- темпом отправки (pacing),
- объёмом данных «в полёте» (inflight),

стремясь поддерживать `BDP = BtlBw × RTprop` [2].

В условиях пакетной балансировки измерения скорости доставки и RTT становятся смесью по нескольким путям. Из-за этого появляются выбросы (например, RTT с перегруженного пути), и базовые оценки BBR могут стать нестабильными. Поэтому для BBR критично не только «переживать» переупорядочивание, но и робастно (устойчиво к выбросам) обрабатывать измерения RTT/скорости доставки.

2.3. TCP-NCR: динамический порог реакции на duplicate ACK

TCP-NCR (Non-Congestion Robustness) улучшает устойчивость TCP к неперегрузочным событиям (в первую очередь — к переупорядочиванию) за счёт динамического порога fast retransmit вместо фиксированных `3 dupACK`.

Порог задаётся как:

$$\mathrm{DupThresh} = \max\left(\mathrm{LT_F} * \left(\frac{\mathrm{InFlightSize}}{\mathrm{SMSS}}\right), 3\right)$$

где:
- `InFlightSize` — текущий объём данных, отправленных, но ещё не подтверждённых (bytes in flight),
- `SMSS` (Sender Maximum Segment Size) — полезная нагрузка TCP-сегмента,
- `LT_F` (Limited Transmit Factor) — константа настройки [4].

Чтобы не «застрять» при реальной потере (когда dupACK может не накопиться), TCP-NCR использует **Extended Limited Transmit**: отправитель продолжает осторожно отправлять новые сегменты в ответ на dupACK. Предложены варианты:
- **Careful Limited Transmit**: 1 новый сегмент на каждые 2 dupACK (быстрее снижает активность при аномалиях),
- **Aggressive Limited Transmit**: 1 новый сегмент на каждый dupACK (лучше удерживает скорость) [4].

TCP-NCR хорошо решает проблему ложных fast retransmit из-за переупорядочивания, но в основном работает с dupACK/потерями и не решает проблему выбросов в измерениях RTT/задержки и других сигналов при многопутевой доставке.

2.4. Lazy TCP (LTCP): “ленивый триггер” и “упрямое окно”

Lazy TCP нацелен на сценарий packet spraying и использует две идеи [5]:

1) **lazy trigger** — откладывает реакцию на dupACK, чтобы не снижать скорость при переупорядочивании.

2) **stubborn window** — различает тип потерь:
- **дискретные потери** (редкие одиночные), характерные для случая, когда отдельный путь деградировал, но остальные пути нормальные — в этом случае `cwnd` не уменьшается;
- **последовательные потери** (несколько подряд), более похожие на перегрузку — в этом случае `cwnd` уменьшается.

Для различения типов потерь используется `SACK` (Selective ACK) — это опция TCP, в которой получатель сообщает, какие блоки данных он уже получил; по ней отправитель может понять, какие сегменты отсутствуют.

Критерий «последовательности» задаётся порогом `k`: если потеряно больше `k` пакетов подряд, потери считаются связанными с перегрузкой (successive loss), иначе — дискретными [5]. `k` трактуется как ожидаемое количество соседних пакетов, которые могли попасть на один перегруженный канал при распылении.

LTCP повышает пропускную способность при packet spraying, но по сути остаётся механизмом вокруг потерь/dupACK и не даёт общего решения для алгоритмов, где основным сигналом является задержка.

2.5. MSwift: устойчивая к пакетной балансировке агрегация сигналов (медиана)

Работа [3] показывает, что одной защиты от dupACK недостаточно: при per-packet балансировке страдают все сигналы перегрузки (например, задержка), потому что измерения приходят как смесь по путям, а реакция на «последнее» измерение делает алгоритм чувствительным к выбросам.

В MSwift используется:

- механизм LTCP для устойчивости к переупорядочиванию,
- и робастная оценка задержки: задержка считается как **медиана** по последним `N` измерениям RTT.

Размер окна:

$$N = \max(\min(\alpha * \mathrm{cwnd}, K), 3)$$

где:

- `α` — параметр отзывчивости (чем меньше, тем медленнее реакция и тем выше устойчивость к выбросам),
- `K` — верхняя граница размера окна,
- `cwnd` — окно перегрузки [3].

Если медианная задержка превышает порог, `cwnd` уменьшается вдвое [3]. За счёт медианы единичный «плохой» путь не должен определять реакцию всего соединения.

2.6. Выводы по обзору и выбор направления для модификации BBR

	TCP-NCR	LTCP	MSwift
Устойчивость к переупорядочиванию пакетов	Да	Да	Да
Устойчивость к перегрузке каналов	Нет	Да	Да
Независимость от стратегии балансировки	Да	Нет	Нет, но можно настроить оптимально для большинства случаев
Отсутствие избыточных потерь при реальной перегрузке	Нет	Нет	Нет

Таблица 1: Сравнение подходов TCP-NCR, LTCP и MSwift

TCP-NCR и LTCP в основном улучшают поведение при переупорядочивании и потерях, но не решают общую проблему выбросов в смешанных многопутевых измерениях (RTT/скорость доставки/ECN и т.п.). Подход MSwift является более общим: он снижает влияние «плохих» измерений за счёт робастной агрегации (медианы) по окну.

Так как BBR критически зависит от корректных оценок `RTprop` и `BtlBw`, наиболее перспективным направлением модификации BBR для пакетной балансировки является внедрение робастной обработки измерений по аналогии с MSwift (медиана по окну недавних измерений), дополняя это механизмами устойчивости к переупорядочиванию из TCP-NCR/LTCP там, где это влияет на измерения и восстановление потерь [2–5].

Литература

1. Dixit A. et al. On the Impact of Packet Spraying in Data Center Networks. 2013.
2. Cardwell N. et al. BBR: Congestion-Based Congestion Control // Queue. 2016.
3. Gerstein B., Silberstein M., Keslassy I. Making congestion control robust to per-packet load balancing in datacenters. 2025.
4. Bhandarkar S. et al. Improving the Robustness of TCP to Non-Congestion Events. 2006.
5. Zhang J., Zhang D., Huang K. Improving datacenter throughput and robustness with Lazy TCP over packet spraying // Computer Communications. 2015.